



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Department of Artificial Intelligence and Machine Learning

AdvisorAI – Intelligent AI-Powered Advisory System Using Agentic RAG

AD23731 Foundations of Agentic AI - Mini-Project

Submitted by

NELSON RAJ I

231501107

SHRIRAM P

231501124

IRSHAN M

231501505

SEPTEMBER 2026



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this mini-project report " **AdvisorAI – Intelligent AI-Powered Advisory System Using Agentic RAG**" is the bonafide work of "Nelson Raj I (231501107), Shriram P(231501124), Irshan M (231501505)" VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in-charge

Submitted to mini-project viva voce examination held on:

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

The Agentic AI-Based AdvisorAI Academic Advising and Decision Support System is a multi-agent academic planning system designed to assist students and academic advisors in making personalized, evidence-grounded course planning decisions by combining student-context analysis, Agentic Retrieval-Augmented Generation (RAG), academic-rule validation, multi-step reasoning, reflection, stateful memory, and Human-in-the-Loop (HITL) validation within a unified architecture. The system operates as a structured agentic workflow in which the advising request and student academic context are first analyzed to identify the student's requirements, completed coursework, remaining requirements, and planning objectives; an agent-driven retrieval component then selectively retrieves relevant academic evidence such as degree requirements, course planning guidance, prerequisite policies, semester credit policies, and academic evidence traceability records; and a recommendation component uses the retrieved evidence together with deterministic academic tools to generate a personalized semester plan. The system incorporates validation tools to check prerequisites, degree requirements, credit limits, and schedule constraints before presenting the recommendation. A reflection or self-correction stage reviews the generated plan against the retrieved evidence and validation results, allowing inconsistencies to be identified and corrected before the recommendation reaches the advisor. Stateful behavior and memory maintain relevant student planning context and previous advising decisions so that subsequent interactions can be handled consistently. The final recommendation is explicitly placed under Human-in-the-Loop control, where an academic advisor can Approve the Plan, Edit and Revalidate it, or Reject it, ensuring that the AI system remains a decision-support tool rather than an autonomous authority. The system also provides an Agentic RAG comparison mechanism that evaluates the same advising scenario using a no-retrieval baseline, fixed Basic RAG, and Agentic RAG, enabling comparison of grounding and reasoning behavior across retrieval strategies. In addition, the evaluation framework supports comparison of different Large Language Models using common advising scenarios and criteria including accuracy, reasoning quality, response consistency, evidence grounding, and constraint satisfaction. The implementation demonstrates a practical, explainable, auditable, and human-supervised approach to AI-assisted academic advising, providing a scalable foundation for personalized curriculum planning and institutional academic decision support.

Keywords: Agentic AI, Academic Advising, Agentic RAG, Retrieval-Augmented Generation, Multi-Agent Systems, Human-in-the-Loop, Large Language Models, Academic Planning, Decision Support System, Reflection, Stateful Memory, Explainable AI.

TABLE OF CONTENTS

Section	Page
1. Introduction	6
1.1 Background and Motivation	6
1.2 Need for Agentic AI	6
1.3 Project Objectives	6
2. Problem Statement and Business Case	7
2.1 Problem Definition	7
2.2 Business / Real-World Context	7
3. User and Stakeholder Analysis	8
3.1 Target Users	8
3.2 Stakeholder Identification	8
3.3 User Requirements	8
4. Proposed Agentic AI Solution	9
4.1 Overall Solution	9
4.2 Key Features	9
4.3 Agent-Based Approach	9
5. Dataset / Knowledge Sources	11
5.1 Dataset Description	11
5.2 Data Flow and Usage	11
6. Tools and Technologies Used	12
6.1 Core Technologies	12
7. Agent Architecture	13
7.1 Agent Components	13
7.2 Agent Memory	13
7.3 Reasoning Flow	13

Section	Page
7.4 Agent Interactions	13
8. Agent Workflow and Orchestration	15
8.1 Workflow Design	15
8.2 Task Decomposition	15
8.3 Agent Coordination	15
8.4 Orchestration Implementation	15
9. Agentic RAG Implementation	16
9.1 Retrieval Architecture	16
9.2 Knowledge Retrieval	16
9.3 Generation Process	16
9.4 Retrieval-Quality Evaluation	16
10. System Implementation	17
10.1 Functional Prototype	17
10.2 Workflow Implementation	17
10.3 Backend API Endpoints	17
10.4 Functional Prototype Screenshots	17
10.5 Source-Code Repository	17
11. Experimental Evaluation and Results	21
11.1 Evaluation Metrics	21
11.2 Testing Methodology	21
11.3 Performance Results	21
11.4 Success-Metric Achievement	21
12. Comparison, Innovation and Scalability	22
12.1 Comparison with Baseline	22
12.2 Advantages of Proposed System	22

Section	Page
12.3 Innovation	22
12.4 Scalability	22
12.5 Real-World Applicability	22
13. Conclusion and Future Work	24
13.1 Conclusion	24
13.2 Future Work	24

1. INTRODUCTION

1.1 Background and Motivation

Academic advising is one of the most important support services in higher education because students continuously need to make decisions related to course selection, prerequisite requirements, degree completion, credit limits, elective choices, semester planning, and academic goals. A student planning a semester may need to consider completed courses, remaining curriculum requirements, course dependencies, available electives, course offerings, timetable constraints, academic performance, and personal interests before selecting an appropriate set of courses. In practice, this information is often distributed across course catalogs, curriculum documents, academic regulations, schedules, and student records, making the advising process highly dependent on manual analysis and individual advisor judgement. As student populations and curriculum structures become more complex, manual academic planning becomes difficult to scale consistently, while students may have limited visibility into the evidence and reasoning behind a recommended academic plan.

Recent advances in Large Language Models (LLMs) — and specifically in Agentic AI, where an LLM is embedded inside a structured, tool-using, multi-step reasoning workflow rather than used as a single question-answering call — make it possible to support academic advising through intelligent and explainable decision assistance. Instead of relying on one large prompt to generate a list of courses, an agentic system can decompose the advising process into specialised stages that understand the student's academic profile, determine the information required to answer the request, retrieve relevant academic knowledge, verify prerequisites and degree constraints, generate a personalised course plan, and reflect on the recommendation before it is presented for human review. Each stage operates on structured shared state so that information produced by one component can be verified and reused by subsequent components, improving traceability and reducing the risk of unsupported recommendations. This project is motivated by the opportunity to apply this Agentic AI pattern to a practical academic-advising problem using Agentic Retrieval-Augmented Generation (RAG), deterministic academic tools, stateful student memory, reflection and self-correction, and Human-in-the-Loop validation, thereby providing an intelligent decision-support system that assists students while retaining the academic advisor as the final decision-maker.

1.2 Need for Agentic AI

Three broad approaches were considered before finalising the design:

- A pure rule-based course selection engine (for example, “if the student has completed Course A, recommend Course B”) is fast and easy to implement but becomes brittle as curriculum structures and student requirements become more complex — it cannot effectively understand natural-language advising requests, reason about different academic goals, adapt to new course descriptions, or provide a detailed explanation for why a particular plan is appropriate.

- A single monolithic LLM prompt that asks for a complete semester plan in one step is more flexible, but it combines student-profile analysis, academic retrieval, prerequisite checking, degree-requirement validation, recommendation generation, and explanation within the same context. This makes errors harder to identify and correct, provides limited control over academic constraints, and creates a higher risk of unsupported or hallucinated course recommendations.
- An Agentic AI pipeline — the approach used here — divides the advising problem into cooperating stages with clearly bounded responsibilities, a shared state object, and sequential dependency. One stage analyses the student's profile and advising request, another determines and retrieves the relevant academic information, the planning stage generates a candidate course plan using retrieved evidence and academic tools, and a separate reflection and validation stage checks the recommendation before it reaches the Human-in-the-Loop advisor review. Each stage therefore consumes structured information produced by the preceding stage rather than independently solving the entire problem.

Agentic AI is therefore not used here as a buzzword; it directly addresses the *grounding, explainability, modularity, self-correction, and auditability* limitations of simpler academic-advising approaches. By combining Agentic RAG, deterministic academic tools, stateful memory, reflection, and Human-in-the-Loop validation, AdvisorAI can produce personalised and evidence-grounded academic plans while keeping the final academic decision under the control of an advisor.

1.3 Project Objectives

ID	Objective
O1	To accept a natural-language academic advising request and automatically understand the student's academic context, including completed courses, remaining requirements, academic interests, and planning goals.
O2	To retrieve relevant academic information such as course descriptions, prerequisite rules, degree requirements, credit policies, and course-planning guidelines using an agent-driven Agentic RAG strategy rather than relying purely on the LLM's internal knowledge.
O3	To automatically generate a personalised and academically feasible semester plan by combining the student's profile, retrieved evidence, and deterministic academic tools for prerequisite, degree-requirement, credit-limit, and schedule validation.
O4	To implement the advising workflow as cooperating AI stages with shared state and sequential dependencies, where the retrieval, planning, validation, and reflection components consume and update verified outputs from previous stages rather than operating as independent function calls.
O5	To implement reflection and self-correction, together with stateful student memory and tool usage, so that the system can identify unsupported recommendations, prerequisite conflicts, missing academic requirements and revise the proposed plan before final review.
O6	To provide a working Human-in-the-Loop academic advising interface in which an academic advisor can Approve the proposed plan, Edit and Revalidate the recommendation, while maintaining the complete recommendation and review history for auditability.

ID	Objective
O7	To evaluate AdvisorAI using common academic-advising scenarios by comparing No-RAG, Basic RAG, and Agentic RAG approaches and by comparing different LLM configurations using measures such as recommendation accuracy, reasoning quality, evidence grounding, response consistency, constraint satisfaction, and response latency.

2. PROBLEM STATEMENT AND BUSINESS CASE

2.1 Problem Definition

Given a student's academic profile and a natural-language advising request such as “What courses should I take next semester?”, the system must automatically understand the student's current academic status, identify the remaining degree requirements, determine which courses are eligible based on completed prerequisites, retrieve relevant academic policies and course information, and generate a personalised and academically feasible course plan. The recommendation must also be checked for prerequisite violations, credit-limit constraints, degree requirements, and other applicable academic rules, while providing a clear explanation of the evidence and reasoning behind the proposed plan. The core technical challenge is that academic advising requires several distinct types of reasoning — student-context understanding, information retrieval, constraint validation, personalised planning, and self-correction — that are difficult to combine reliably within a single prompt or conventional rule-based system and must remain consistent, traceable, and auditable across different students and advising scenarios.

2.2 Business / Real-World Context

In the academic advising context, students often need to make course-planning decisions using information distributed across curriculum structures, course catalogs, prerequisite rules, degree requirements, credit policies, semester offerings, and individual academic records. For example, a student may select an advanced course because it matches their interests without realizing that a prerequisite course has not yet been completed, or may choose several courses that exceed the permitted credit load or conflict with other academic requirements. These issues can result in unsuitable semester plans, repeated advisor interventions, delayed academic progress, and uncertainty for students about whether their course choices support their degree and career goals. A system that analyses student context, retrieves relevant academic evidence, validates constraints, and explains why a course plan has been recommended has direct value for:

- Students, who receive personalised, evidence-grounded course recommendations together with explanations of prerequisites, degree requirements, and other constraints affecting their academic plan.

- Academic advisors, who can use the AI-generated plan as a structured first-pass recommendation and focus human attention on exceptional, ambiguous, or high-impact cases while retaining final approval authority.
- Departments and curriculum administrators, who can maintain consistent academic rules and course information and reduce errors caused by incomplete or outdated manual interpretation of curriculum requirements. This project reproduces that business problem at a mini-project scale using a synthetic-but-realistic 3,000-record Tamil Nadu grievance dataset, so that the classification and prioritisation logic can be benchmarked against genuine historical patterns rather than being invented from scratch for each new complaint.

This project reproduces that academic-advising problem at a mini-project scale using structured student profiles, course information, prerequisite rules, degree requirements, academic policies, and advising knowledge sources, allowing the recommendation and validation workflow to be evaluated across representative student-planning scenarios rather than generating course suggestions solely from the language model's general knowledge.

3. USER AND STAKEHOLDER ANALYSIS

3.1 Target Users

- Students — use the AdvisorAI interface to submit an academic advising request in natural language and expect a clear, personalised outcome: which courses are recommended, why they are suitable, what prerequisites or degree requirements are satisfied, and what constraints were considered.
- Academic advisors — use the Human-in-the-Loop review interface to examine the proposed academic plan, inspect the retrieved evidence and validation results, and Approve, Edit & Revalidate, or Reject the recommendation when necessary.
- Department / Curriculum administrators— maintain and review course information, prerequisite rules, degree requirements, credit policies, and other academic knowledge sources used by the retrieval and validation components.
- Academic reviewers and evaluators — need to see the multi-stage Agentic AI reasoning explicitly, including the student-context analysis, Agentic RAG retrieval, recommendation generation, reflection/validation, and Human-in-the-Loop decision rather than only the final course plan.

3.2 Stakeholder Identification

Stakeholder	Interest / Concern	How the System Addresses It
Student	Wants a personalised, accurate, and understandable academic plan.	Provides a structured course recommendation with student-specific context, retrieved academic evidence, prerequisite checks, degree requirements, and explanations
Academic Advisor	Wants to review recommendations and retain final academic decision-making authority.	Human-in-the-Loop interface allows the advisor to Approve, Edit & Revalidate, or Reject the proposed academic plan.
Department / Curriculum Administrator	Wants accurate and consistent use of course, prerequisite, curriculum, and credit information.	Analytics dashboard: totals, priority distribution, category / department / location breakdown
Project Reviewer / Evaluator	Wants to see genuine agentic reasoning, not a black box	Displays the Agentic RAG evidence, agent workflow, validation/reflection results, Human-in-the-Loop stage, and RAG/model evaluation controls.
System Developer / Maintainer	Wants a modular and extensible system that is easy to test and improve.	Separates agents, retrieval, academic tools, state/memory, evaluation, and the Streamlit interface into independent components.

3.3 User Requirements

- A student must be able to submit an academic advising request in natural language using a simple text interface, optionally selecting their student profile so that AdvisorAI can access relevant academic context such as completed courses, current semester, remaining requirements, and academic goals.
- The system must show visible progress through the advising workflow, including student-context analysis, Agentic RAG retrieval, recommendation generation, and reflection/validation, so that the multi-agent nature of the decision is not hidden behind a single loading indicator.
- The final result must be understandable without technical knowledge: the student should see the proposed courses, credit load, reasons for the recommendations, retrieved academic evidence, prerequisite and degree-rule validation, and the overall validation status.

- An academic advisor must be able to review the complete recommendation and supporting evidence through the Human-in-the-Loop interface, with explicit options to Approve the Plan, Edit & Revalidate, or Reject the recommendation.
- The system must maintain relevant student and advising history so that previous recommendations, advisor decisions, and feedback can be reused in subsequent interactions without requiring the student to provide the same information repeatedly.
- The system must not fail silently — if a retrieval operation, LLM call, academic tool, or supporting data source is unavailable, the system must provide a clear status or fallback behaviour rather than presenting an unsupported recommendation as a valid final plan.

4. PROPOSED AGENTIC AI SOLUTION

4.1 Overall Solution

The proposed solution is a multi-agent, state-based academic advising pipeline that takes a student's natural-language advising request together with their academic profile and produces a structured, personalised, and explainable academic plan. Conceptually, the flow moves from student profile and query intake, through student-context analysis, Agentic RAG-based academic retrieval, course recommendation, academic-rule validation, and reflection, to a final advising decision that is presented to an academic advisor for Human-in-the-Loop approval, editing and revalidation, or rejection. The approved decision and relevant advising information are then persisted as student memory for future academic-planning interactions.

The interface is divided into two main panels. The left panel, titled 'Student', contains a 'Student ID' dropdown menu with 'STU001' selected, an 'Advising request' text area with the placeholder 'What courses should I take next semester?', and a red 'Generate Academic Plan' button. The right panel, titled 'AdvisorAI' with a graduation cap icon, displays a 'Multi-Agent Academic Advising System with Agentic RAG'. It shows a table with columns 'GPA', 'Completed', 'Iterations', and 'Major'. The values are: GPA 8.2, Completed 4, Iterations 1, and Major Computer Science. Below this is a section titled 'Proposed Course Plan — Arun Kumar' containing three green bars for courses CS210, CS301, and CS310. A 'Retrieved Academic Evidence' section is empty. A 'Validator' section with a green checkmark icon shows the message 'Prerequisites, degree logic, and schedule checks passed.' At the bottom, a 'Human-in-the-Loop Approval' section states 'The plan is NOT final until an academic advisor reviews it.' and features three buttons: 'Approve Plan' (green checkmark), 'Edit & Revalidate' (pencil icon), and 'Reject' (red X icon).

Figure 4.1. High-level agentic academic advising pipeline implemented in AdvisorAI

4.2 Key Features

- Specialised academic-advising agents with clearly defined responsibilities, including student-context analysis, Agentic RAG-based academic retrieval, course planning, and reflection/validation, without unnecessary agent sprawl.
- A shared, state-based context object that carries the student's profile, advising request, retrieved evidence, tool results, proposed plan, validation feedback, and Human-in-the-Loop status from one stage to the next, so that each recommendation is traceable to the information produced earlier in the workflow.
- Agent-driven academic grounding: instead of relying only on the LLM's internal knowledge, the Retrieval Agent determines what information is needed and retrieves relevant course descriptions, prerequisite rules, degree requirements, credit policies, and academic guidance from the AdvisorAI knowledge base.
- Structured outputs at every major stage, so that student context, retrieved evidence, course recommendations, validation results, reflection feedback, and advisor decisions remain machine-readable and can be passed reliably between components.
- Deterministic academic-tool validation for important constraints such as prerequisite eligibility, remaining degree requirements, credit limits, and schedule conflicts, reducing dependence on free-form LLM reasoning for rule-based academic decisions.
- Reflection and self-correction: the Validator/Reflection stage independently checks the proposed academic plan against retrieved evidence and academic constraints. When an inconsistency or unsupported recommendation is detected, the workflow can revise the plan before it reaches the advisor.
- Stateful student memory: relevant student information, previous recommendations, advisor decisions, and feedback can be persisted so that subsequent advising interactions can use prior context instead of starting from an empty state.
- Human-in-the-Loop validation: the final academic plan is presented to an authorized advisor with explicit Approve Plan, Edit & Revalidate, and Reject actions. Edited recommendations are checked again before they can be approved.
- Transparent execution and evidence display: the interface exposes the retrieved Agentic RAG evidence, validation status, reflection results, and recommendation details so that the system's decision process is visible rather than hidden behind a single final answer.
- Baseline and model evaluation support: the system provides a comparison workflow for No-RAG, Basic RAG, and Agentic RAG, while also supporting comparison of different LLM configurations using common academic-advising scenarios and evaluation metrics.

4.3 Agent-Based Approach

The agents in AdvisorAI are deliberately scoped to represent genuinely different academic-advising reasoning tasks rather than being an arbitrary split of one recommendation into multiple prompts:

- The Student Context and Intent Agent performs profile understanding and request interpretation — a task that benefits from structured, factual processing of the student's completed courses, current academic status, interests, goals, and natural-language advising request. Its output provides the clean academic context required by the subsequent stages rather than repeatedly analysing the raw student input.
- The Agentic Retrieval and Planning Agent performs evidence retrieval and academic plan generation — a task that depends on the structured student context and on retrieving relevant course descriptions, prerequisite rules, degree requirements, credit policies, and other academic guidance. The agent determines what information is needed, retrieves supporting evidence, and combines that evidence with deterministic academic-tool results to construct a personalised set of candidate courses.
- The Reflection and Validation Agent performs independent checking and self-correction — a task that depends on the student's structured context, the retrieved evidence, the proposed course plan, and the results of academic validation tools. It checks for prerequisite violations, degree or credit inconsistencies, unsupported recommendations, and other conflicts, and can send the plan back for correction before it reaches the Human-in-the-Loop review stage.

Each stage therefore genuinely depends on the structured output of the previous stage. This sequential dependency distinguishes AdvisorAI from simply running three unrelated recommendation functions in parallel, because the final academic plan is constructed from verified student context, retrieved evidence, academic-rule checks, and an explicit reflection step.

Student

Student ID

STU001

Advising request

What courses should I take next semester?

Generate Academic Plan

Validator

Prerequisites, degree logic, and schedule checks passed.

Human-in-the-Loop Approval

The plan is NOT final until an academic advisor reviews it.

PLAN APPROVED — Advisor approval recorded.

Approve Plan

Edit & Revalidate

Reject

Audit / Agent State

```

{
  "draft_plan": [
    0: "CS220",
    1: "CS381",
    2: "CS320"
  ],
  "validation": {
    "valid": true,
    "errors": []
  },
  "iterations": 1,
  "approval": "approved"
}

```

5. DATASET / KNOWLEDGE SOURCES

5.1 Dataset Description

The AdvisorAI project uses a set of structured local datasets and academic knowledge sources to provide the grounding information required for personalised course planning and academic validation. Instead of relying on a single large dataset, the system combines *student academic profiles, course catalogue information, and academic-policy knowledge* so that different agents can access the information required for context understanding, retrieval, recommendation, and validation. These sources are loaded programmatically by the application and are used as the internal grounding layer for the Agentic RAG workflow.

Column	Description	Observed Values
students.json	Student academic profile dataset	Student identifiers, programme/academic context, completed coursework, academic interests, goals, and planning-related information used to construct the student's current academic state
catalog.json	Course catalogue and curriculum information	Course codes, course names, credit information, prerequisite relationships, and course-level details used for eligibility checking and semester planning
knowledge_base.json	Academic advising and policy knowledge source	Degree requirements, prerequisite policies, semester credit policies, course-planning guidance, and academic evidence used for retrieval-grounded recommendations and explanations

Table 5.1. Structure and purpose of the student, course catalogue, and academic knowledge sources used by AdvisorAI

The three sources serve different but complementary purposes within the agentic workflow. The student profile dataset provides the factual academic context required by the Student Context and Intent Agent, including the student's completed coursework, current status, interests, and planning objectives. The course catalogue provides structured course information and prerequisite relationships that can be consumed by the planning and validation stages. The academic knowledge base contains the policy and guidance information needed to explain why a course is relevant and to support checks involving degree requirements, credit limits, prerequisite rules, and semester-planning constraints.

These sources are not treated as passive reference files. They form the grounding layer for the Agentic RAG pipeline, where the retrieval stage determines what academic information is required for the current advising request and retrieves the corresponding evidence before the recommendation is generated. The retrieved evidence is then passed through the planning and validation stages, allowing the final course plan to be connected to explicit academic information rather than being generated only from the language model's general knowledge.

A further design consideration is that different academic facts are stored in different forms. Student information and course information are represented as structured records, while advising rules and policy guidance are represented as retrievable knowledge entries. AdvisorAI therefore combines structured data lookup, agent-driven retrieval, and deterministic academic validation tools rather than assuming that one data format is sufficient for the complete advising task. This separation also improves auditability because the system can distinguish between student-specific facts, course-specific information, and general academic policies when presenting the evidence behind a recommendation.

The knowledge sources are intentionally kept local and lightweight for the mini-project implementation. This makes the system reproducible and allows the Agentic RAG workflow, reflection stage, Human-in-the-Loop validation, and baseline comparison to be evaluated without depending on an external institutional database. In a production deployment, the same architecture could be connected to a live university course catalogue, curriculum database, student information system, academic regulations, and semester schedule sources

5.2 Data Flow and Usage

- Student profile selection and academic context — the student profile data is loaded when an advising case is started and provides the system with the student's completed courses, current academic status, interests, goals, and other planning-related context. This information is passed to the Student Context and Intent Agent to establish the factual state of the advising request.

- **Agentic academic knowledge retrieval** — for every advising request, the retrieval stage analyses the student's query and academic context to determine what information is required. It then performs multiple targeted searches across the academic knowledge base and course catalogue to retrieve relevant course descriptions, prerequisite rules, degree requirements, credit policies, and course-planning guidance. The retrieved evidence is injected into the planning stage rather than relying only on the language model's general knowledge.
- **Supporting recommendation and validation decisions** — the retrieved academic evidence is combined with structured course and student information by the planning stage to generate a candidate semester plan. Deterministic academic tools are then used to check prerequisite eligibility, remaining degree requirements, credit limits, and other applicable constraints. The reflection and validation stage reviews these results and can trigger a correction when the proposed plan is unsupported or inconsistent with the available academic evidence.
- **Stateful memory and advising history** — relevant student context, generated recommendations, validation results, and Human-in-the-Loop decisions are maintained as structured application state and persistent advising history. This allows subsequent advising interactions to reuse previously established academic context and decisions instead of treating every request as an entirely independent query.
- **Basic system analytics and evidence traceability** — the system exposes the retrieved evidence, proposed courses, validation results, reflection outcome, and Human-in-the-Loop status through the interface so that a reviewer can inspect how the final recommendation was produced. The implementation also supports baseline comparison between No-RAG, Basic RAG, and Agentic RAG approaches for evaluating the effect of retrieval strategy on academic planning.

6. TOOLS AND TECHNOLOGIES USED

6.1 Core Technologies

Layer	Technology	Purpose in This Project
Programming Language	Python 3	Implements the AdvisorAI application, agent pipeline, retrieval logic, academic validation tools, state management, and persistence components
LLM / Reasoning Layer	LLM-compatible agent pipeline	Supports the academic context understanding, retrieval-grounded planning, reflection, and validation stages of the multi-agent workflow; the local mini-project can also run using the included deterministic fallback logic without an external API key

Layer	Technology	Purpose in This Project
Agent Orchestration	Explicit sequential state-based agent pipeline	Implements the Student Context → Agentic RAG Retrieval / Planning → Reflection & Validation workflow using shared structured state between stages
Frontend / Application U	Streamlit	Provides the interactive academic-advising dashboard for student selection, natural-language advising requests, recommendation display, evidence inspection, validation, HITL actions, and baseline comparison
Data Processing	pandas	Supports structured data handling and lightweight processing of academic records and evaluation-related tabular information
Database (secondary)	MongoDB, via docker-compose.yml	Document-oriented persistence matching the original specification's storage schema
Knowledge Representation	JSON	Stores the local student profiles, course catalogue, and academic knowledge-base entries used by the advising workflow

The implementation deliberately avoids unnecessary external orchestration infrastructure for the mini-project. Instead, the explicit state-based Python agent pipeline acts as the central orchestration mechanism, connecting the student data, academic knowledge sources, retrieval component, deterministic validation tools, reflection stage, persistent memory, and Human-in-the-Loop interface into one controlled workflow. The architecture is therefore designed around the project requirements of multi-agent interaction, Agentic RAG, tool usage, reflection and self-correction, stateful memory, academic-rule validation, and Human-in-the-Loop approval. This also keeps the implementation lightweight and reproducible while providing a clear path for replacing the local retrieval and reasoning components with production-scale institutional services or external LLM APIs in a future version.

7. AGENT ARCHITECTURE

7.1 Agent Components

The AdvisorAI system consists of three specialised agent components: the Student Context and Intent Agent, the Agentic Retrieval and Planning Agent, and the Reflection and Validation Agent. Each component has a distinct academic-advising responsibility, receives the structured state produced by the previous stage, uses the appropriate academic data or validation tools, and produces a structured output for the next stage. The components are connected through a shared state-based orchestration environment so that student context, retrieved evidence,

recommendations, validation results, and reflection outcomes remain available throughout the complete advising workflow.

Agent	Goal	Input	Tools / Data	Structured Output
Agent 1 — Student Context & Intent	Understand the student's academic situation and convert the natural-language advising request into structured planning requirements	Selected student profile + raw advising request	Student profile data; course-completion records; academic interests and goals; structured context-processing logic	student_context, completed_courses, remaining_requirements, interests, goals, advising_intent
Agent 2 — Agent Retrieval & Planning	Determine the academic information required, retrieve relevant evidence, and generate a personalised candidate semester plan	Agent 1 structured context + advising request	Agent-driven multi-query retrieval; course catalogue; academic knowledge base; prerequisite and degree information; credit-policy guidance; academic planning tools	retrieved_evidence, proposed_courses, credit_load, planning_reasons, evidence_trace
Agent 3 — Reflection & Validation	Check the proposed plan against academic evidence and constraints, identify inconsistencies, and correct the recommendation when required	Agent 1 + Agent 2 JSON (combined)	Prerequisite validator; degree-requirement checks; credit-limit checks; retrieved evidence; reflection/self-correction logic	validation_status, constraint_results, reflection_findings, corrected_plan, final_recommendation

Table 7.1. Role, goal, input, tools, and structured output for each of the three agents.

7.2 Agent Memory

The AdvisorAI system uses two forms of memory, both intentionally lightweight and appropriate for a mini-project implementation. Working, or short-term, memory is represented by the shared structured state object that is carried through the complete advising pipeline for the duration of a single request; this allows Agent 2 to access the student context and intent produced by Agent 1, while Agent 3 can access the student context, retrieved evidence, proposed plan, and validation results produced by the preceding stages. The shared state therefore acts as the common working memory connecting the three specialised agents.

Long-term, or episodic, memory is implemented using SQLite-based persistence. Relevant student profiles, advising cases, generated recommendations, validation outcomes, and Human-in-the-Loop decisions can be stored and retrieved for subsequent interactions. This allows the system to retain meaningful academic-planning context rather than treating every advising request as completely independent. Previous recommendations and decisions can therefore contribute to continuity when the same student returns for another advising interaction.

The memory design is deliberately separated from the agent reasoning logic so that student information, retrieval evidence, planning results, reflection outcomes, and advisor decisions can be persisted without coupling them to a single agent. In addition to supporting stateful behaviour, this persistence contributes to auditability and traceability, because the system can retain the information associated with how an academic recommendation was generated and subsequently approved, edited, revalidated, or rejected.

Unlike the reference grievance system, AdvisorAI does **not** treat every request as permanently stateless because academic advising is inherently longitudinal: a student's completed courses, remaining requirements, previous planning decisions, and advisor feedback can remain relevant to future semester planning. The memory mechanism is therefore included as an explicit project capability while remaining lightweight enough for local execution and demonstration.

7.3 Reasoning Flow

Agent 1 — Student Context Analysis

1. Receive the selected student profile, advising request, and relevant academic records.
2. Analyse the request and create a structured context containing completed courses, remaining requirements, interests, and goals.
3. Validate the context against student and course records.
4. Store the verified context in the shared state for Agent 2.

Agent 2 — Agentic Retrieval and Planning

1. Receive Agent 1's structured context and advising intent.
2. Identify required academic information and generate targeted retrieval queries.
3. Use Agentic RAG and academic tools to retrieve course, prerequisite, requirement, and credit information.
4. Generate a personalised semester plan with proposed courses, credit load, reasons, and supporting evidence.

Agent 3 — Reflection and Validation

1. Receive the combined student context, retrieved evidence, proposed plan, and validation results.
2. Check prerequisites, degree requirements, credit limits, and evidence consistency.
3. Reflect on identified issues and revise the plan when necessary.

4. Produce a final validation result for the advisor's Approve Plan, Edit & Revalidate, or Reject decision.

Overall flow: Student understanding → Agentic retrieval & planning → Reflection & validation. Each agent uses the structured state from the previous stage, maintaining complete context and traceability throughout the workflow.

7.4 Agent Interactions

Interaction between AdvisorAI agents is strictly sequential and state-mediated. Agent 2 receives the structured student context and advising intent generated by Agent 1, while Agent 3 receives the proposed plan, retrieved evidence, and validation information from Agent 2.

This one-directional structured hand-off forms the agent-to-agent communication mechanism. The shared state includes student context, completed courses, remaining requirements, retrieved evidence, proposed courses, credit load, validation results, and reflection findings.

The pipeline enforces this dependency: Agent 1 establishes the academic context, Agent 2 performs Agentic RAG retrieval and generates the candidate plan, and Agent 3 performs reflection and validation using the accumulated state. This ensures traceability and prevents later agents from bypassing earlier stages.

Finally, the **Human-in-the-Loop** review receives the recommended courses along with the supporting evidence, validation results, and reflection outcome.

8. AGENT WORKFLOW AND ORCHESTRATION

8.1 Workflow Design

The end-to-end AdvisorAI workflow follows a strict state-based sequential pipeline: START first activates the Student Context and Intent Agent, which analyses the selected student profile and natural-language advising request and writes the structured academic context into the shared state. This state then feeds the Agentic Retrieval and Planning Agent, which reads the student context, determines the information required for the advising task, retrieves relevant academic evidence, and writes the proposed course plan and supporting evidence into the shared state. The resulting state then feeds the Reflection and Validation Agent, which reads the accumulated student context, retrieved evidence, proposed plan, and academic-tool results and performs validation and self-correction. After the reflection and validation stage, a finalize step assembles the complete advising result, including the recommended courses, credit load, retrieved evidence, validation status, reflection findings, and Human-in-the-Loop review status, before the workflow reaches END. The resulting architecture ensures that each agent operates on the output of the preceding stage and that the complete academic decision-support process remains traceable through the shared state.

START → Student Context & Intent Agent → Agentic Retrieval & Planning Agent → Reflection & Validation Agent → Finalize → END.

8.2 Task Decomposition

AdvisorAI divides academic planning into three independent stages:

- Student Context and Intent: Understands the student profile, academic records, and advising request.
- Agentic Retrieval and Planning: Retrieves relevant academic evidence and creates a candidate semester plan.
- Reflection and Validation: Checks constraints, identifies errors, and corrects the plan before review.
- This decomposition enables each stage to be tested, improved, or replaced independently while maintaining a clear evaluation process.

8.3 Agent Coordination

AdvisorAI uses a shared state to coordinate the three agents sequentially.

Student Context → Retrieval & Planning → Reflection & Validation Each stage reads the previous stage's structured output and adds its own results to the shared state. The final state contains the student context, retrieved evidence, proposed plan, validation results, reflection findings, and Human-in-the-Loop status, providing a clear and auditable workflow

8.4 Orchestration Implementation

The AdvisorAI project follows an incremental integration methodology in which the major capabilities of the advising system are developed as focused, self-contained components before being connected into the complete agent workflow. The student-context processing, Agentic RAG retrieval, academic planning, deterministic validation tools, reflection logic, state persistence, and Human-in-the-Loop interface are developed and tested as individual capabilities and are then integrated through the shared state-based orchestration pipeline.

This approach helps isolate failures to specific stages such as student-context processing, retrieval queries, academic-rule validation, recommendation generation, or reflection and correction, rather than treating the complete advising workflow as a single black-box operation. Once the individual stages are stable, they are connected sequentially so that the output of one stage becomes the structured input for the next stage, while the shared state preserves the complete execution context.

The orchestration implementation therefore provides both functional automation and visible engineering representation of the Agentic AI workflow. During execution, the interface can expose the progression through the Student Context, Agentic RAG Retrieval and Planning, and Reflection and Validation stages, together with the retrieved evidence, validation findings, and final Human-

in-the-Loop status. This makes the internal agent workflow inspectable during demonstration and supports systematic testing and evaluation of the complete academic-advising pipeline.

9. AGENTIC RAG IMPLEMENTATION

9.1 Retrieval Architecture

AdvisorAI uses a lightweight, in-process Agentic RAG system over a local academic knowledge base and course catalogue, avoiding external vector databases or embedding services.

The retrieval process:

- Analyses the student context and identifies required academic information.
- Generates multiple targeted queries based on the advising intent.
- Searches and ranks relevant academic records using term-overlap matching.
- Combines and filters the retrieved evidence.
- Checks whether the evidence is sufficient and performs additional retrieval when needed.
- Passes the final evidence to the planning stage along with student data and academic-tool results.
- This approach provides adaptive, traceable, and locally deployable retrieval while keeping the system lightweight

9.2 Knowledge Retrieval

Consuming Agent	What Is Retrieved	Why
Agent 1	Student profile information, completed courses, remaining academic requirements, interests, goals, and relevant academic	Provides the factual student context required to interpret the advising request and ensures that later planning is based on the student's actual academic situation rather than a generic profile

Consuming Agent	What Is Retrieved	Why
Agent 2	Relevant course information, prerequisite rules, degree requirements, semester credit policies, course-planning guidance, and supporting academic evidence	Allows the planning stage to construct an evidence-grounded course recommendation and verify that proposed courses are relevant and academically feasible
Agent 3	Indirect: Agent 1's structured student context and Agent 2's retrieved academic evidence, proposed plan, and validation results	Agent 3 does not independently retrieve the same information; it reasons over the already-grounded upstream state to identify inconsistencies, validate constraints, and perform reflection/self-correction

9.3 Generation Process

Generation in AdvisorAI follows a structured, stage-wise process rather than a single text-generation call. The planning stage uses verified student context, retrieved academic evidence, and deterministic tool results to create a candidate academic plan with proposed courses, credit load, and supporting reasons. The Reflection and Validation stage then checks the plan for consistency, accuracy, and academic constraints before presenting it for Human-in-the-Loop review. The system uses structured state objects to ensure reliable data flow, traceability, and reproducibility. The current mini-project runs locally without an external LLM API key, while its modular design allows future LLM integration without affecting the existing retrieval, validation, and human-review controls.

9.4 Retrieval-Quality Evaluation

AdvisorAI's retrieval quality is evaluated qualitatively using representative academic-advising scenarios. The evaluation checks whether the Agentic RAG component retrieves relevant course records, prerequisites, degree requirements, credit policies, and planning guidelines for each student request.

The system is also evaluated on its ability to perform additional targeted searches when the initial retrieved evidence is insufficient, distinguishing it from a single fixed retrieval query.

A current limitation is that AdvisorAI uses keyword-overlap matching instead of semantic embeddings or a vector database. Therefore, paraphrased queries may not retrieve all relevant information. For example, "subjects I can register for after completing my programming foundation" may not match records containing the formal course or prerequisite terms.

This limitation is documented as a retrieval constraint. As a future enhancement, the keyword-based retriever can be replaced or combined with a sentence-embedding-based semantic index, enabling retrieval based on meaning rather than exact keyword matches

10. SYSTEM IMPLEMENTATION

10.1 Functional Prototype

AdvisorAI is a lightweight, locally runnable Python and Streamlit prototype rather than a monolithic application. It includes a Streamlit interface for student selection, advising requests, plan generation, evidence inspection, Human-in-the-Loop validation, and baseline comparison. The Python agent pipeline handles student context and intent analysis, Agentic RAG retrieval and planning, and reflection and validation. Local JSON files provide student, course, and academic knowledge, while SQLite supports persistent student and advising-case state. Deterministic checks validate prerequisites, degree requirements, and credit constraints. The repository is organised into separate interface, agent, retrieval, storage, data, and evaluation modules, allowing the complete workflow to run locally without external database or orchestration services.

10.2 Workflow Implementation

AdvisorAI is implemented as a lightweight, locally runnable application that integrates the multi-agent pipeline, Agentic RAG, academic validation, persistent memory, and Human-in-the-Loop review. The Streamlit interface allows students to select their profiles, submit advising requests, and view generated academic plans, retrieved evidence, validation results, and approval options.

The Python-based agent pipeline connects the three stages—Student Context and Intent, Agentic Retrieval and Planning, and Reflection and Validation—through a shared structured state object. The system retrieves student profiles, course catalogues, and academic knowledge from local JSON files, while SQLite stores student information and previous advising decisions for future interactions. The complete workflow proceeds from student selection and advising requests through retrieval, planning, validation, and final Human-in-the-Loop review, ensuring a reproducible and reliable local implementation without requiring external infrastructure.

10.3 Backend API Endpoints

Method	Endpoint	Description
POST	Generate Academic Plan	Accepts the selected student and natural-language advising request and runs the Student Context → Agentic Retrieval & Planning → Reflection & Validation workflow to generate a personalised academic plan

Method	Endpoint	Description
GET	Student / Case History	Retrieves previously stored student and advising-case information from the persistent application store for stateful academic-planning interactions
GET	Case Details	Displays the complete advising result for a selected case, including student context, retrieved academic evidence, proposed courses, validation results, reflection findings, and HITL status
GET	Knowledge / Evidence View	Returns the academic evidence retrieved for the current advising case, including course information, prerequisite rules, degree requirements, credit policies, and planning guidance
GET	Edit & Revalidate	Accepts an advisor-edited course plan and runs the academic validation and reflection stages again before presenting the updated result
GET	HITL Decision	Records the advisor's Approve Plan or Reject decision together with the current recommendation and validation state
GET	RAG Comparison	Runs the selected advising scenario through the available No-RAG, Basic RAG, and Agentic RAG comparison workflow and displays the resulting comparison information

10.4 Functional Prototype Screenshots

The Streamlit dashboard is the main interface of AdvisorAI, allowing users to select a student profile, enter an academic advising request, and generate a personalized academic plan. The prototype displays the proposed courses, credit load, supporting Agentic RAG evidence, and validation results, making the planning process transparent and traceable. It also provides Human-in-the-Loop controls to approve, edit and revalidate, or reject the recommendation, along with a RAG Comparison function to evaluate different retrieval approaches.

10.5 Source-Code Repository

The current implementation spans two backend tracks and a shared React frontend rather than a single standalone application. The repository contains the agent modules, the LangGraph and Node orchestration code, the FastAPI and Express API layers, the dataset file, the Docker Compose

configuration for MongoDB, and the environment-variable template, and should be included as a GitHub or equivalent link in the final submitted report.

GITHUB LINK: <https://github.com/siva25900/Agenti-Ai-mini-project>

11. EXPERIMENTAL EVALUATION AND RESULTS

11.1 Evaluation Metrics

- AdvisorAI is evaluated based on academic context correctness, retrieval relevance, and academic plan correctness to ensure that recommendations match student requirements and satisfy prerequisite, degree, and credit constraints. Reasoning quality and reflection effectiveness are assessed by checking whether the system provides evidence-based explanations, identifies inconsistencies, and corrects errors. Human-in-the-Loop validation measures the advisor's ability to review, approve, edit, or reject plans, while stateful consistency checks whether student information and previous decisions are retained across interactions.
- The evaluation also compares Agentic RAG with No-RAG and Basic RAG approaches to assess academic grounding. Resilience measures the system's ability to operate using local and deterministic fallbacks when optional dependencies are unavailable, while end-to-end execution time measures the total time taken to generate, validate, and present an academic plan.

11.2 Testing Methodology

AdvisorAI was tested using representative academic-advising scenarios covering completed courses, remaining degree requirements, prerequisites, credit limits, and student goals. Each scenario was executed through the complete workflow, from student-context analysis and Agentic RAG retrieval to academic planning, reflection, validation, and Human-in-the-Loop review. The generated student context, retrieved evidence, proposed plans, and validation results were examined to assess academic correctness, retrieval relevance, constraint satisfaction, and error correction. The system's local fallback behaviour was also tested to ensure that the workflow could continue without an external LLM service. Finally, the same scenarios were used to compare No-RAG, Basic RAG, and Agentic RAG approaches based on academic grounding, reasoning quality, consistency, resilience, and execution performance

11.3 Success-Metric Achievement

The AdvisorAI prototype successfully integrates Student Context and Intent, Agentic RAG Retrieval and Planning, Reflection and Validation, and Human-in-the-Loop review through a shared state-based pipeline. It generates structured academic recommendations

using student profiles, course information, prerequisites, degree requirements, and credit policies, with deterministic validation to identify potential conflicts.

The system provides visible academic evidence through targeted Agentic RAG retrieval, supports reflection and correction, and allows advisors to approve, edit and revalidate, or reject recommendations. It also maintains structured state and persistent advising information using SQLite. The local fallback mechanism enables the prototype to operate without an external LLM API key. Although the system supports No-RAG, Basic RAG, and Agentic RAG comparisons, numerical performance and multi-LLM results remain subject to further experimental testing.

12. COMPARISON, INNOVATION AND SCALABILITY

12.1 Advantages of Proposed System

- Explainable by construction — Shows student context, evidence, recommendations, validation results, and advisor decisions for clear reasoning.
- Grounded in academic knowledge — Uses course data, prerequisites, degree requirements, credit rules, and planning guidance through Agentic RAG.
- Constraint-aware and self-correcting — Checks prerequisites, requirements, and credits, while reflection identifies and corrects inconsistencies.
- Human-supervised by design — Advisors can Approve, Edit & Revalidate, or Reject recommendations after reviewing the evidence.
- Modular and auditable — Each agent has a defined role, making components easy to update and track independently.
- Stateful and reusable — Stores student profiles, advising cases, recommendations, and validation results for future use.
- Lightweight and reproducible — Uses local data, lightweight retrieval, SQLite, and Streamlit for easy local deployment.

12.2 Innovation

The main innovation of AdvisorAI lies in applying the Agentic AI approach to academic advising through a structured, evidence-grounded workflow rather than a single text-generation process. It separates advising into three stages: Student Context and Intent, Agentic RAG-based Retrieval and Planning, and Reflection and Validation, ensuring clear responsibilities and reliable recommendations. AdvisorAI further integrates Agentic RAG, deterministic academic validation, self-correction, persistent student memory, and Human-in-the-Loop approval into a single pipeline. Its lightweight implementation using Python, SQLite, local knowledge sources, and Streamlit allows the complete workflow to run without external LLM API requirements. Thus, its innovation lies in combining these capabilities into a transparent, traceable, and human-supervised academic decision-support system.

12.3 Scalability

AdvisorAI is designed with a modular architecture that allows future expansion from local execution to a multi-user deployment. Its retrieval layer can be upgraded to semantic search as academic knowledge sources grow, while SQLite can be replaced with a production database to support more students and concurrent advising requests.

The shared-state architecture also allows additional agents, such as Semester Scheduling, Career Alignment, and Academic Progress Monitoring, to be integrated without changing the existing workflow. Furthermore, local JSON knowledge sources can be connected to institutional systems, including course catalogues, student information systems, and academic regulations. This provides a flexible foundation for developing AdvisorAI into a larger academic-advising platform.

12.4 Real-World Applicability

With the AdvisorAI prototype already implementing the core academic-advising workflow — student-context analysis, Agentic RAG retrieval, course planning, academic-rule validation, reflection, stateful memory, and Human-in-the-Loop review — integration into a real institutional advising environment would primarily involve connecting the existing workflow to the university's operational data and student-service channels rather than redesigning the complete reasoning architecture. A production deployment could receive advising requests through an existing student portal, mobile application, advisor dashboard, or other institutional interface and connect the same processing flow to live student records, course catalogues, curriculum structures, semester schedules, prerequisite databases, and academic regulations.

In such an environment, the system could operate as a decision-support layer for both students and academic advisors: student requests would be interpreted using the student's current academic record, relevant institutional knowledge would be retrieved, a candidate semester plan would be generated and validated, and the recommendation would then be presented to an advisor for final review. The existing modular design also supports integration with institutional authentication, student-information systems, curriculum-management platforms, and notification services, making the prototype a practical foundation for a larger academic-advising application beyond the classroom.

13. CONCLUSION AND FUTURE WORK

13.1 Conclusion

This project successfully demonstrates AdvisorAI, an Agentic AI-based academic advising system that combines a multi-agent architecture, Agentic RAG, structured state management, academic validation, and Human-in-the-Loop review. The three agents work together to understand student requirements, retrieve relevant academic evidence, generate personalized semester plans, and validate recommendations against prerequisites, degree requirements, and credit constraints.

The prototype integrates reflection and self-correction, persistent memory, and advisor approval to provide transparent and evidence-grounded recommendations. Its lightweight implementation using Python, Streamlit, local knowledge sources, and SQLite makes it suitable for local demonstration without requiring an external LLM API key. Overall, AdvisorAI demonstrates the potential of Agentic AI as an explainable and human-supervised academic decision-support system, with scope for future integration of external LLMs, institutional databases, and advanced retrieval technologies.

13.2 Future Work

1. Semantic and Hybrid Agentic RAG – Replace keyword-based retrieval with semantic search using sentence embeddings, metadata filtering, and agent-generated queries for better retrieval of academic information.
2. Live Institutional Data Integration – Connect AdvisorAI with live university sources such as course catalogues, curriculum regulations, prerequisite databases, timetables, course availability, and student records.
3. Multi-Semester Degree Planning – Extend planning across multiple semesters by considering prerequisites, credit limits, electives, course availability, and graduation requirements.
4. Personalised Academic Pathways – Consider career interests, specialisations, elective preferences, workload, and long-term goals to generate personalised academic pathways.
5. Continuous Academic Progress Monitoring – Compare actual student progress with approved plans, detect changes in requirements or course availability, and identify cases needing advisor attention.
6. Multi-LLM Evaluation – Compare different LLMs using academic accuracy, reasoning quality, evidence grounding, consistency, constraint satisfaction, and response latency.
7. Multilingual Academic Advising – Support English, Tamil, and mixed-language/Tanglish academic queries while improving intent and constraint understanding.